CertificateAuthority

System Design Description

Abstract
This document describes the design and implementation of the CertificateAuthority support system of Arrowhead (documentation edition 5.3.0) (`ah5-CertificateAuthority-java-spring`).

Contents

1 Documentation edition and implementation scope **3**

2 Overview **3**

3 Implementation **4**
3.1 Implementation language and tools 4
3.2 Functional properties implementation 4
3.3 Non functional properties implementation 5
3.4 Deployment 5
3.5 Local modifications 6
3.6 Verification 6

4 Services **7**

5 Go and Rust implementation variants **7**
5.1 Go implementation 7
5.2 Rust implementation 7

6 References **8**

7 Revision History **9**
7.1 Amendments 9
7.2 Quality Assurance 9

1 Documentation edition and implementation scope

This is the 5.3.0 documentation edition, dated 2026-11-09. It carries forward the system and service contracts from the retained 5.2.2 documentation and identifies the current primary implementation sources. The former `documentation/v5.2.2` folder is retained unchanged.

Document version 5.3.0 is distinct from executable, dependency, registered system/service and wire-contract versions. Values still defined by the source (including 5.2.2 artefact versions, 1.0.0 service versions and `SysMonitor-AMQP-IDD-5.2.2` payload identifiers) are preserved. This documentation update does not change runtime behaviour or establish a new implementation release. Historical verification statements describe the tests recorded in the inherited baseline; this revision validates document compilation and references.

2 Overview

This document describes the CertificateAuthority system implementation, which realizes the CertificateAuthority SysD [1]. The CertificateAuthority is the local-cloud CA: it signs certificates under the Arrowhead profiles (onboarding, device, system, operator), validates certificates and trusted keys, keeps the certificate register and publishes the CRL. In Section 3, we describe implementation details of the system.

3 Implementation

This implementation realizes the CertificateAuthority SysD in `ah5-CertificateAuthority-java-spring`. Since 8 November 2026 it follows the Arrowhead Java-Spring pattern of the other support systems (for example Blacklist and TimeManager). The sources, a private copy of the Arrowhead common library (`vendor/`), tests, images and the deployment definition are owned by this folder. The earlier native implementation (JDK `HttpsServer`, SQLite) is no longer built; `legacy/` keeps the older history.

3.1 Implementation language and tools

- Java 21, Spring Boot 3.4, the Arrowhead common library (private source copy), Bouncy Castle for X.509, Hibernate/JPA with the MariaDB driver, Log4j2. Maven produces `arrowhead-certificate-authority-5.2.2.jar`; the entry point is `CertificateAuthorityMain`.
- Storage: a private MySQL schema `ah_certificate_authority` in the system's own database container.

3.2 Functional properties implementation

The following components implement the system:

Component	Responsibility
`CertificateAuthority SystemInfo`	System and service models (registered by the common `ApplicationInitListener` with `CERT_AUTH` interfaces).
`CertificateAuthorityAPI`, `GeneralManagementAPI`	HTTP operations; every path is served with and without the base path `/certificate-authority`.
`CertificateSigningService`	`sign`: CSR checks, caller profiles, reserved names, name ownership, validity limits, certificate issuing with `ca=false`.
`CertificateValidation Service`	`check-certificate`, `check-trusted-key`.
`CertificateManagement Service`	Certificate register, revocation (by id or content), trusted keys.
`CrlService`	CRL generation (`get-crl`), publication of `<crl.dir>/ca.crl` after each revocation and periodically.
`CallerProfileFilter`	Admits only system (`sy`) and operator (`op`) client certificates.
`CertificateAuthorityStatus Service`	`status` document and the sysMonitor echo detail.

Every request passes the common filter chain: the outsourced authentication filter verifies the identity token with Authentication and binds it to the client certificate; `CallerProfileFilter` checks the certificate profile; the management filter admits Sysop, CertificateAuthority and Onboarding to `/mgmt/` paths. The services then apply the CA's identity rules: Onboarding, Sysop and CertificateAuthority may sign (with profile limits), Sysop and CertificateAuthority manage, and Onboarding may revoke only the certificates it requested. The Blacklist filter is not used, so the CA keeps working while Blacklist is unavailable.

3.2.1 Data handled and stored

Schema `ah_certificate_authority`:

Tables	Content
`issued_certificate`	Certificate register: serial number, common name, requesting identity, validity, revocation time, PEM.
`trusted_key`	Trusted public keys with validity; a SHA-256 of the PEM keeps them unique.
`logs`	Operational log entries (`get-log`).

The management responses keep the field names of the native implementation (for example `serial_number`, `revoked_at`). `scripts/migrate-sqlite-to-mysql.py` imports a native SQLite store, keeping the ids.

3.3 Non functional properties implementation

3.3.1 Security

The HTTPS server requires TLS 1.2/1.3 with client certificates, checks them against the cloud CRL (`arrowhead.ssl.crl-file`) and accepts only certificate-bound identity tokens. The common `Security ProfileGuard` refuses to start with TLS disabled or declared identities unless a development profile is selected explicitly. The issuer certificate and signing key are PEM files mounted read-only; only this system mounts the key. The database account holds data-manipulation privileges on its own schema only. Errors use the Arrowhead `ErrorMessageDTO`.

3.3.2 Power management

None; the system is a server process without power states. The `sysMonitor power` observation always reports line supply.

3.3.3 Internal monitoring

Logs go to stdout, a rolling file and the `logs` table. The common `SysMonitorPublisher` publishes echo, power and status every 30 s with the routing key `<cloud>.sysmonitor.CertificateAuthority.certificateAuthority`; the echo detail carries the certificate counts.

3.3.4 Configuration

Configuration is `config/application.properties`, with environment placeholders for the cloud name, organization, registry, broker and CRL timing; secrets come from `/run/secrets` (`configtree`). The CA-specific keys are `ca.certificate.file`, `ca.key.file`, `crl.dir`, `crl.validity.hours`, `crl.refresh.seconds`, `cert.validity.days`, `onboarding.cert.validity.hours` and `reserved.names`.

3.4 Deployment

The local `Dockerfile` builds only this folder and runs `arrowhead-certificate-authority-standalone:5.2.2`. `compose.yaml` adds the private database `certificate-authority-db` (MySQL 8.4, TLS required, internal network, no host port). Its init script creates the schema and restricts the application account to `SELECT, INSERT, UPDATE, DELETE`. The HTTPS API uses port 8448. The application runs as UID 10001 with a read-only root filesystem; only the CRL output directory is writable. This packaging is supplied for operator deployment; existing test-cloud deployments are not changed by it.

ServiceRegistry is the bootstrap endpoint; Authentication and DynamicServiceOrchestration are looked up there. AMQPS monitoring is a network dependency. The CRL output directory must be distributed to the other systems of the cloud by the operator.

3.5 Local modifications

Notes on this implementation in the workspace:

- Native Java port (2026-10-16) of the Python CA; Java-Spring implementation (2026-11-08).
- Reserved names; LocalCloudManager no longer signs (J6); CRL publication (H3).
- Since the Spring implementation, every operation (including `check-certificate`, `get-crl` and `/health`) needs a certificate-bound identity token, and `generalManagement` adds `get-log` and `get-config`.

3.6 Verification

The JUnit tests cover the signing rules, revocation, CRL content and publication, certificate and trusted-key validation, the caller profile check, and every 5.2.2 route with and without the base path. An end-to-end test runs the whole application (filters, controllers, JPA) against a fresh MySQL schema created by the deployment init script with the restricted account. Startup with the development sample is refused by the security profile guard. A run in a live cloud with Onboarding is not part of this verification.

4 Services

The implementation of the services is based on the SD and IDD documents referenced below.

Services produced	SysD ref	SD ref	IDD ref
`certificateSigning`	[1]	[2]	[3]
`certificateValidation`	[1]	[4]	[5]
`certificateAuthority Management`	[1]	[6]	[7]
`sysMonitor`	[1]	[8]	[9]
Services consumed	**SysD ref**	**SD ref**	**IDD ref**
systemDiscovery, serviceDiscovery	[10]	[11]	–
identity	[12]	[13]	–
sysMonitor (publisher)	AMQP broker (SysMonitor)	–	–

5 Go and Rust implementation variants

5.1 Go implementation

Primary system folder: `Eclipse-Arrowhead-go/ah5-CertificateAuthority-go`.

The system owns its Go module, launcher and build configuration. Use the per-system `Makefile` and `go.mod`; the containing Go tree is the build source, not a historical Arrowhead copy. Ordinary native roles use independent identities and JSON snapshot state rather than the Java database. Do not assume Java SQL migration, clustering or identical transaction/retry semantics. The registry has its own configuration and optional native AMQP monitor; other roles must be checked against their own registrations.

Listening port. Built-in default 127.0.0.1:8448 (Arrowhead default). The shipped config.json, compose.yaml and Dockerfile use the same port on all interfaces ("listen": "0.0.0.0:8448"; 8540–8555 layout before 2026-11-09). Override with "listen" in the configuration file, SERVER_PORT=N or -server.port N; the port overrides keep the configured host.

Native behavior and outstanding work. Role/profile-constrained CSR signing, chain checks, certificate revocation, signed CRL export and trusted keys. Java SQL import, automatic CRL-file publication, wider relying-party revocation enforcement, HSM/OCSP/HA. These are the boundaries recorded by the native compatibility document; later implementation changes still require direct qualification. Native design/route evidence: `Eclipse-Arrowhead-go/ah5-CertificateAuthority-go/documentation/CertificateAuthority`.

For this Java interface, select only a native transport and operation explicitly implemented and advertised by that port. A matching method/path does not establish DTO, error, authorization, retry or persistence equivalence. Java MQTT/AMQP bindings are not automatically provided by a Go or Rust role. Differential contract and security checks remain necessary before substitution.

5.2 Rust implementation

Primary system folder: `Eclipse-Arrowhead-rust/ah5-CertificateAuthority-rust`.

The per-system entry point is declared as a binary of `arrowhead-support-rust` in `ah5-support-runtime-rust/Cargo.toml`. Build from the primary Rust workspace with the applicable package/binary selection. Documentation links can resolve into the shared core/support catalogue. Independent process identity and state remain required.

The ordinary native profile uses Tokio/Axum and rustls with peer-identity checks and independent locked JSON state. Network effects are not distributed database transactions. The precise trust, CRL reload and persistence guarantees are those of the native profile, not the Java implementation.

Listening port. Default 127.0.0.1:8448 (Arrowhead default, role certificateauthority). Override with "listen": "host:port" in the configuration, SERVER_PORT=N or –server.port N; the port overrides keep the configured host.

Native behavior and outstanding work. Native CSR verification/signing, profiles, revocation, PEM CRLs, trusted keys. Explicit SANs only (no CSR extension copying); automatic key renewal/root rollover, OCSP/HSM/HA, full validity/DTO normalization. These are the boundaries recorded by the native compatibility document; later implementation changes still require direct qualification. Native design/route evidence: `Eclipse-Arrowhead-rust/ah5-support-runtime-rust/documentation/CertificateAuthority`.

For this Java interface, select only a native transport and operation explicitly implemented and advertised by that port. A matching method/path does not establish DTO, error, authorization, retry or persistence equivalence. Java MQTT/AMQP bindings are not automatically provided by a Go or Rust role. Differential contract and security checks remain necessary before substitution.

6 References

[1] J. Delsing, “CertificateAuthority – System Description,” Eclipse Arrowhead, SysD 5.3.0, 2026, ah5-CertificateAuthority-java-spring/documentation/v5.3.0/CertificateAuthority-v5.3.0-SysD.pdf.

[2] ——, “certificateSigning – Service Description (CertificateAuthority),” Eclipse Arrowhead, SD 5.3.0, 2026, certificateAuthority-v5.3.0-certificateSigning-SD.pdf.

[3] ——, “certificateSigning HTTP/TLS/JSON – Interface Design Description (CertificateAuthority),” Eclipse Arrowhead, IDD 5.3.0, 2026, certificateAuthority-v5.3.0-certificateSigning-HTTP-IDD.pdf.

[4] ——, “certificateValidation – Service Description (CertificateAuthority),” Eclipse Arrowhead, SD 5.3.0, 2026, certificateAuthority-v5.3.0-certificateValidation-SD.pdf.

[5] ——, “certificateValidation HTTP/TLS/JSON – Interface Design Description (CertificateAuthority),” Eclipse Arrowhead, IDD 5.3.0, 2026, certificateAuthority-v5.3.0-certificateValidation-HTTP-IDD.pdf.

[6] ——, “certificateAuthorityManagement – Service Description (CertificateAuthority),” Eclipse Arrowhead, SD 5.3.0, 2026, certificateAuthority-v5.3.0-certificateAuthorityManagement-SD.pdf.

[7] ——, “certificateAuthorityManagement HTTP/TLS/JSON – Interface Design Description (CertificateAuthority),” Eclipse Arrowhead, IDD 5.3.0, 2026, certificateAuthority-v5.3.0-certificateAuthorityManagement-HTTP-IDD.pdf.

[8] ——, “sysMonitor – Service Description (CertificateAuthority),” Eclipse Arrowhead, SD 5.3.0, 2026, certificateAuthority-v5.3.0-sysMonitor-SD.pdf.

[9] ——, “sysMonitor AMQP/TLS/JSON – Interface Design Description (CertificateAuthority),” Eclipse Arrowhead, IDD 5.3.0, 2026, certificateAuthority-v5.3.0-sysMonitor-AMQP-IDD.pdf.

[10] ——, “ServiceRegistry – System Description,” Eclipse Arrowhead, SysD 5.3.0, 2026, ah5-core-java-spring/serviceregistry/documentation/v5.3.0/ServiceRegistry-v5.3.0-SysD.pdf.

[11] ——, “systemDiscovery – Service Description (ServiceRegistry),” Eclipse Arrowhead, SD 5.3.0, 2026, ah5-core-java-spring/serviceregistry/documentation/v5.3.0/ServiceRegistry-v5.3.0-systemDiscovery-SD.pdf.

[12] ——, “Authentication – System Description,” Eclipse Arrowhead, SysD 5.3.0, 2026, ah5-core-java-spring/authentication/documentation/v5.3.0/Authentication-v5.3.0-SysD.pdf.

[13] ——, “identity – Service Description (Authentication),” Eclipse Arrowhead, SD 5.3.0, 2026, ah5-core-java-spring/authentication/documentation/v5.3.0/Authentication-v5.3.0-identity-SD.pdf.

7 Revision History

7.1 Amendments

No.	Date	Version	Subject of Amendments	Author
1	2026-10-22	5.2.2	Document created from the Arrowhead LaTeX templates for the 5.2.2 Java Spring implementation, including the local extensions of this workspace.	Jerker Delsing
2	2026-10-25	5.2.2	Independent Java build, private helpers and state, per-system deployment and configuration.	Codex (draft)
3	2026-10-30	5.2.2	Checked against the primary Java source tree (Eclipse-Arrowhead-java): standalone deployment, configuration and security statements updated.	Claude Opus 5.5 (draft)
4	2026-11-08	5.2.2	Java-Spring implementation: common library, filter chain, JPA on a private MySQL schema, migration from SQLite.	Claude Opus 5.5 (draft)
5	2026-11-09	5.3.0	New documentation edition; primary-source references and Go/Rust implementation scope; 5.2.2 retained.	Documentation maintenance

7.2 Quality Assurance

No.	Date	Version	Approved by
1		5.3.0	